

EcoSprout CarbonEngine Mathematical Review & Improvement Report

Executive Summary

This report reviews the current mathematical framework used by the EcoSprout CarbonEngine to estimate carbon footprints for e-commerce products, flights, and food delivery orders. The objective was to identify inaccuracies, systematic biases, and opportunities for improvement while preserving the lightweight, browser-based architecture of the system.

Overall Assessment

Component	Current Rating	Improved Rating
Product Estimation	6/10	8.5/10
Flight Estimation	8/10	9/10
Food Estimation	5/10	8/10
Overall Engine	6.5/10	8.5/10

The current system is suitable for an MVP and hackathon demonstration, but several mathematical assumptions can produce large overestimations or underestimations. Most issues stem from multiplicative factors that unintentionally double-count emissions.

1. Product Footprint Analysis

Current Methodology

Current product emissions are calculated using:

```
Total CO2e =  
(Base Category Footprint × Material Multiplier × Weight Multiplier)  
+ Shipping Emissions
```

or

```
Total CO2e =  
(Price × 0.40)  
+ Shipping Emissions
```

when category detection fails.

Identified Problems

1.1 Material Multiplier Causes Significant Overestimation

Current formula:

```
MaterialMultiplier =  
AverageMaterialFactor / 3.0
```

with bounds:

```
0.3 ≤ MaterialMultiplier ≤ 3.0
```

Example:

```
Laptop Base = 250 kg  
  
Aluminum Factor = 8.2  
  
Multiplier = 8.2 / 3  
            = 2.73  
  
Result = 250 × 2.73  
        = 682.5 kg CO2e
```

This is unrealistic because category footprints already incorporate material impacts.

Root Cause

The material multiplier amplifies a value that already contains embedded material emissions, resulting in double counting.

1.2 Weight Scaling Creates Artificial Discontinuities

Current logic:

```
Weight < 0.2kg → 0.5x  
Weight < 1kg   → 0.8x  
Weight > 20kg  → 1.5x  
Weight > 50kg  → 2.0x
```

Example:

```
19.9kg Television = 1.0x  
20.1kg Television = 1.5x
```

A difference of 200 grams produces a 50% increase in emissions.

This introduces instability and inconsistent estimates.

1.3 Price-Based Fallback Is Weak

Current fallback:

```
CO2e = Price × 0.40
```

This assumes price and carbon footprint are directly proportional.

Examples where this fails:

- Luxury watch
- Designer clothing
- Limited edition collectibles
- Artwork

These products may be expensive but have relatively small physical footprints.

Recommended Improvements

Material Adjustment

Replace:

$\text{MaterialFactor} / 3$

with:

$$\begin{aligned} \text{MaterialAdjustment} = \\ 1 + ((\text{MaterialFactor} - 3) / 20) \end{aligned}$$

Examples:

Material	Current	Proposed
Plastic	1.00	1.00
Steel	0.63	0.95
Aluminum	2.73	1.26

This preserves influence while preventing runaway values.

Weight Adjustment

Replace tiered thresholds with:

$$\begin{aligned} \text{WeightAdjustment} = \\ 1 + (0.15 \times \log_{10}(\text{weightKg} + 1)) \end{aligned}$$

Examples:

Weight	Adjustment
0.2kg	1.01
1kg	1.05
10kg	1.16
50kg	1.26

This creates smooth scaling.

Improved Price Fallback

Replace:

$$\text{Price} \times 0.40$$

with:

$$5 \times \text{Price}^{0.7}$$

Benefits:

- Reduces distortion from luxury goods
- Maintains correlation with manufacturing complexity
- Prevents extreme values

Recommended Product Formula

$$\begin{aligned} \text{Total CO}_2\text{e} = & \\ & \text{BaseCategoryFootprint} \\ & \times \text{MaterialAdjustment} \\ & \times \text{WeightAdjustment} \\ & + \text{ShippingEmissions} \end{aligned}$$

Recommended multiplier range:

$$0.8 \leq \text{Adjustment} \leq 1.3$$

rather than:

$$0.3 \leq \text{Adjustment} \leq 3.0$$

2. Flight Footprint Analysis

Current Methodology

$$\begin{aligned} \text{BaseFootprint} = & \\ & \text{Distance} \times \text{DistanceFactor} \end{aligned}$$

with:

Distance	Factor
<1500km	0.18
<4000km	0.13
≥4000km	0.11

Then:

```
LayoverPenalty =
(Base × 0.08 × N)
+ (35 × N)
```

Finally:

```
Total =
(Base + LayoverPenalty)
× CabinMultiplier
```

Identified Problems

2.1 Step-Based Distance Factors

Flights immediately change emission factors when crossing arbitrary thresholds.

Example:

```
1499 km → 0.18
1501 km → 0.13
```

This creates unrealistic jumps.

2.2 Layover Penalty Double Counts Emissions

Current calculation includes:

1. Percentage penalty
2. Fixed takeoff penalty

Both attempt to model the same phenomenon.

This inflates emissions for multi-leg journeys.

2.3 Missing Radiative Forcing

Aircraft produce warming effects beyond direct CO₂ emissions through:

- Contrails
- Nitrogen oxides
- High-altitude atmospheric interactions

Most aviation carbon calculators account for this using a radiative forcing multiplier.

Recommended Improvements

Dynamic Distance Factor

Use:

$$\text{Factor} = 0.22 - (0.000025 \times \text{Distance})$$

bounded by:

$$0.09 \leq \text{Factor} \leq 0.22$$

This creates smooth transitions.

Simplified Layover Model

Replace:

$$\begin{aligned} &(\text{Base} \times 0.08 \times N) \\ &+ (35 \times N) \end{aligned}$$

with:

$$25 \times N$$

or

$$\text{Base} \times (1 + 0.10 \times N)$$

but not both.

Add Radiative Forcing

Introduce:

$$\text{RadiativeMultiplier} = 1.9$$

to reflect total climate impact.

Recommended Flight Formula

$$\begin{aligned} \text{DistanceEmissions} &= \\ \text{Distance} &\times \text{DynamicFactor} \end{aligned}$$

$$\begin{aligned} \text{Total CO}_2\text{e} &= \\ &(\text{DistanceEmissions} \\ &+ 25 \times \text{Layovers}) \\ &\times \text{CabinMultiplier} \\ &\times 1.9 \end{aligned}$$

3. Food Delivery Analysis

Current Methodology

Food items are assigned flat emissions:

Category	Current
Red Meat	6.5
Seafood	2.2
Poultry	1.8

Category	Current
Dairy	1.0
Plant-Based	0.5

Then:

Total =
Factor × Quantity

Identified Problems

3.1 Portion Size Ignored

Current model treats:

Single Burger

and

Double Burger

similarly.

Portion size is one of the strongest predictors of food emissions.

3.2 Dairy Underestimated

Cheese and other dairy products often have footprints closer to meat than vegetables.

Current values systematically underestimate dairy-heavy meals.

3.3 Delivery Emissions Missing

The delivery journey itself contributes emissions.

Ignoring delivery omits a major component of the user experience.

Recommended Improvements

Introduce Serving Multipliers

Examples:

Item Type	Multiplier
Small	0.8
Regular	1.0
Large	1.4
Double	1.7
Family	4.0

Revised Food Factors

Category	Proposed
Plant-Based	0.4
Dairy	1.8
Poultry	2.0
Seafood	2.5
Red Meat	6.5

Add Delivery Impact

Vehicle	Emissions
Bicycle	0.1 kg
Motorcycle	0.4 kg
Car	1.0 kg

Distance-based scaling may be added later.

Recommended Food Formula

```
Total CO2e =  
ProteinFactor  
× ServingMultiplier  
× Quantity  
+  
DeliveryImpact
```

4. Confidence Scoring System

Recommendation

Every estimate should include a confidence rating.

High Confidence

Requirements:

- Category identified
- Material identified
- Weight identified

Medium Confidence

Requirements:

- Category identified
- Partial metadata available

Low Confidence

Requirements:

- Price-only fallback
 - Missing category
 - Missing route information
-

Example Output

Estimated Carbon Footprint:

142 kg CO₂e

Confidence:

High

Data Sources:

- ✓ Category Detected
- ✓ Material Detected
- ✓ Weight Detected

This significantly improves user trust and transparency.

Final Recommendation

The current CarbonEngine architecture is fundamentally sound and suitable for browser-extension deployment. The primary objective should not be adding complexity but improving the stability and realism of existing heuristics.

The highest-priority improvements are:

1. Replace material multipliers with bounded adjustment factors.
2. Replace threshold-based weight scaling with logarithmic scaling.
3. Replace flight distance brackets with a continuous emission model.
4. Add radiative forcing to aviation calculations.
5. Add serving-size and delivery adjustments to food calculations.
6. Introduce confidence scoring across all estimate types.

Implementing these changes is expected to substantially improve estimate consistency, reduce extreme outliers, and increase user trust without requiring external APIs or large lifecycle assessment databases.